KU LEUVEN

FACULTEIT INDUSTRIELE
INGENIEURSWETENSCHAPPEN

TECHNOLOGIECAMPUS GENT

Geavanceerde Computerarchitectuur

Project: beeldverwerking (image processing)
MEI-ecfs

Charlotte BERT
Sander HAUSTRAETE

Academiejaar 2025 - 2026

1 Context

Voor het project richten we ons op digitale beeldverwerking (image processing). Het doel is om enkele beeldverwerkingstechnieken te implementeren en de uitvoering op GPU en CPU met elkaar te vergelijken. Er werden drie toepassingen geïmplementeerd:

1. **Randdetectie (Edge Detection):** hierbij worden scherpe overgangen in pixelintensiteit gedetecteerd om contouren in het beeld zichtbaar te maken.
2. **Bewegingsdetectie (Motion Detection):** hierbij worden opeenvolgende frames met elkaar vergeleken om bewegende objecten te detecteren en aan te duiden.
3. **Voorgrondextractie (Foreground Extraction):** hierbij wordt een binair masker opgesteld dat de voorgrond van de achtergrond onderscheidt, op basis van het verschil tussen de huidige frame en een dynamisch achtergrondmodel. De voorgrond blijft scherp, terwijl de achtergrond vervaagd wordt.

Voor elke toepassing wordt er een CPU-implementatie met OpenCV vergeleken met een GPU-implementatie met PyCUDA.

2 Theoretische achtergrond

Beeldverwerking is een zeer breed onderwerp. Daarom hebben wij ons beperkt tot 3 toepassingen. Deze toepassingen steunen op het manipuleren van pixelmatrices. Omdat digitale beelden bestaan uit honderdduizenden tot miljoenen onafhankelijke datapunten, vereisen deze operaties aanzienlijke rekenkracht, waardoor deze toepassingen interessant zijn voor parallelle verwerking. Hieronder volgt de theoretische basis van de drie bestudeerde toepassingen.

2.1 Randdetectie (Edge Detection)

Randdetectie (Edge Detection) lokaliseert de grenzen van objecten door abrupte veranderingen in pixelintensiteit te identificeren. Omdat randdetectie zeer gevoelig is voor beeldruis, wordt er eerst Gaussian blur toegepast op de afbeelding. Voor dit project is er gekozen om de Sobel-operator te implementeren een veelgebruikte gradiënt-gebaseerde methode.

De Sobel-operator berekent de ruimtelijke gradiënt van de beeldintensiteit om zo de locaties met de grootste intensiteitstoename (van donker naar licht) te detecteren. Dit wordt gerealiseerd door middel van convolutie, waarbij twee 3×3 kernels over de afbeeldingsmatrix worden geschoven: één voor de detectie van horizontale veranderingen (G_x) en één voor verticale veranderingen (G_y):

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Zodra deze convolutie is uitgevoerd, wordt de gradiëntmagnitude (de daadwerkelijke randsterkte) per pixel berekend. Normaliter wordt volgende formule gebruikt:

$$\text{Magnitude} = \sqrt{G_x^2 + G_y^2}$$

Maar om de duurdere wortelberekening te vermijden wordt gebruik gemaakt van:

$$\text{Magnitude} = |G_x| + |G_y| \tag{1}$$

De output is een grijswaarden edge-map waarbij hogere pixelwaarden overeenkomen met sterkere randen.

De formules zijn overgenomen van GeeksforGeeks [3]

2.2 Bewegingsdetectie (Motion Detection)

Bewegingsdetectie (Motion Detection) lokaliseert veranderende objecten in een video door de verschillen tussen opeenvolgende beelden (frames) te identificeren. Net als bij randdetectie is deze techniek erg gevoelig voor kleine schommelingen in belichting en cameraruis. Daarom wordt elk inkomend frame doorgaans eerst omgezet naar grijswaarden. Voor dit project is er gekozen om beeldverschilbewerking (frame differencing) te implementeren, een fundamentele en veelgebruikte methode voor bewegingsdetectie.

Deze methode werkt door het huidige beeld van de video te vergelijken met het vorige beeld. Het algoritme berekent het absolute wiskundige verschil in pixelintensiteit tussen deze twee frames. Hierdoor worden statische, stilstaande objecten genegeerd en lichten uitsluitend de pixels op waar verandering heeft plaatsgevonden.

Wiskundig gezien wordt dit gerealiseerd door de intensiteitswaarde van elke pixel in het huidige frame (F_t) af te trekken van de corresponderende pixel in het voorgaande frame (F_{t-1}). Om negatieve pixelwaarden te voorkomen wordt de absolute waarde berekend. Dit resulteert in een zogeheten verschilsmatrix of delta-frame (ΔF):

$$\Delta F(x,y) = |F_t(x,y) - F_{t-1}(x,y)| \tag{2}$$

Om uiteindelijk tot een bruikbaar resultaat te komen waarbij onbelangrijke veranderingen (zoals kleine schaduwen of ruis) worden gefilterd, wordt de berekende verschilsmatrix vergeleken met een drempelwaarde (threshold). Pixels met een intensiteitsverschil boven deze drempel worden geclassificeerd als beweging en krijgen de maximale pixelwaarde. De overige pixels worden als statisch beschouwd en op de minimale waarde gezet.

Om deze gedetecteerde beweging visueel duidelijk te maken, is een laatste stap nodig. Het algoritme zoekt in de opgeschoonde zwart-wit afbeelding naar contouren. Om te voorkomen dat onbeduidende veranderingen, zoals lichte ruis, als beweging worden gemarkeerd, wordt er een minimale oppervlakte (area) ingesteld. Voor elke contour die groot genoeg is, berekent de software

de uiterste coördinaten om er een rechthoekig kader (een zogenaamde bounding box) omheen te trekken. Dit kader wordt ten slotte over het oorspronkelijke videobeeld geprojecteerd.

De theorie is gebaseerd op de implementatie van GeeksforGeeks [2].

2.3 Voorgrondextractie (Foreground Extraction)

Voorgrondextractie (Foreground detection) is een beeldverwerkingstechniek waarbij actieve objecten op de voorgrond worden gescheiden van de achtergrond. In dit project gebeurt dit op basis van een dynamisch achtergrondmodel.

Voor elke pixel wordt een achtergrondwaarde bijgehouden. De huidige frame F_t wordt vergeleken met deze achtergrond B_t. Wanneer het absolute verschil groter is dan een ingestelde drempelwaarde, wordt de pixel als voorgrond geclassificeerd:

$$D(x,y) = |F_t(x,y) - B_t(x,y)|$$

$$M(x,y) = \begin{cases} 255, & \text{als } D(x,y) > T \\ 0, & \text{anders} \end{cases}$$

Het achtergrondmodel wordt per frame geüpdatet met een moving average:

$$B_{t+1}(x,y) = (1-\alpha)B_t(x,y) + \alpha F_t(x,y)$$

Hierbij bepaalt α hoe snel het achtergrondmodel zich aanpast aan veranderingen in opeenvolgende beelden. Het resulterende binair masker wordt gebruikt om de voorgrond scherp te houden en de achtergrond te vervagen.

Gebruik gemaakt van wiki [1].

3 Methodologie

3.1 Randdetectie (Edge Detection)

Voor de implementatie van randdetectie werd de Sobel-operator gebruikt, zie sectie 2.1.

In de eerste stap wordt elk frame omgezet naar grijswaarden om de computationele complexiteit te verminderen en enkel luminantie-informatie te behouden. Vervolgens werd een Gaussian blur toegepast om beeldruis te reduceren en kleine lokale variaties te onderdrukken. Dit verbetert de stabiliteit.

Daarna werd de Sobel-operator toegepast via convolutie met twee 3×3 kernels, respectievelijk voor horizontale (G_x) en verticale (G_y) gradiënten. Voor de CPU-implementatie werden OpenCV-

functies gebruikt. Voor de GPU-implementatie werden eigen PyCUDA-kernels geschreven voor de Gaussian blur en Sobel-bewerking.

Na het berekenen van de gradiënten werd voor elke pixel de magnitude van de intensiteitsverandering bepaald. Op basis van deze magnitude werd een grijswaarden randafbeelding gegenereerd, waarbij helderdere pixels overeenkomen met sterkere randen.

3.2 Bewegingsdetectie (Motion Detection)

Voor bewegingsdetectie werd gebruikgemaakt van frame differencing, waarbij opeenvolgende videoframes met elkaar worden vergeleken om veranderingen in de scène te identificeren.

Elk frame werd eerst omgezet naar grijswaarden om de verwerking te vereenvoudigen. Vervolgens werd het absolute verschil tussen het huidige frame (F_t) en het vorige frame (F_{t-1}) berekend, resulterend in een verschilsmatrix (ΔF).

Deze matrix werd vervolgens omgezet naar een binaire afbeelding door toepassing van een drempelwaarde. Hierdoor werden enkel significante veranderingen behouden, terwijl ruis en kleine fluctuaties werden onderdrukt.

Om het resultaat verder te verfijnen werden operaties uit OpenCV toegepast (morphological opening, closing en dilatatie). Deze stappen verwijderen kleine artefacten en vullen onderbroken gebieden in het masker op.

Op basis van het verfijnde binaire masker werden vervolgens contouren gedetecteerd. Om irrelevante detecties te vermijden werd een minimale oppervlakte meegegeven. Voor elke geldige contour werd de kleinste omhullende rechthoek (bounding box) berekend.

Deze bounding boxes werden uiteindelijk geprojecteerd op het originele videobeeld om de bewegende objecten visueel te lokaliseren.

3.3 Voorgrondextractie (Foreground Extraction)

Voor de extractie van de voorgrond werd een dynamisch achtergrondmodel gebruikt. Elke frame werd vergeleken met dit achtergrondmodel. Pixels waarvan het verschil groter was dan een ingestelde drempelwaarde werden opgenomen in een binair voorgrondmasker.

Pixels met waarde 255 in het masker behoren tot de voorgrond en blijven ongewijzigd. Pixels met waarde 0 worden beschouwd als achtergrondpixels. Op deze achtergrond werd een Gaussian blur toegepast. Daarna werden het scherpe voorgrondbeeld en de vervaagde achtergrond opnieuw gecombineerd.

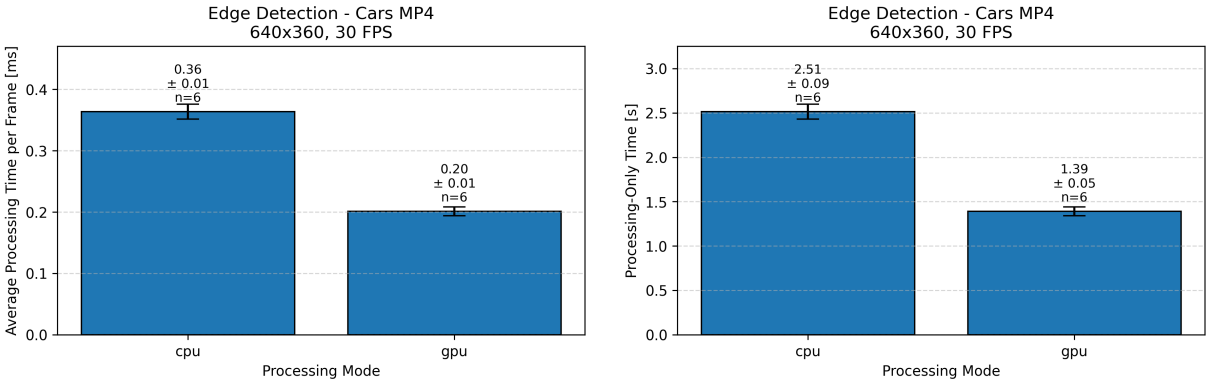
4 Resultaten en observaties

De metingen werden genomen met een vaste block size van 16x16 op een RTX Ada 500 mobile GPU. Voor de CPU metingen werd een Intel Core Ultra 7 155H gebruikt met 16 cores en 22 threads. Er werden voor de metingen 3 soorten videos bekeken: een lage workload, met een resolutie van 640x360, een middelgrote workload, met een resolutie van 1920x1080, en een grote workload, met een resolutie van 3840x2160.

Er worden verschillende metingen genomen, waaronder de totale tijd, de totale verwerkingstijd, de gemiddelde FPS op basis van de totale tijd en op basis van de totale verwerkingstijd en de gemiddelde verwerkingstijd per frame. Van deze metingen wordt enkel dieper gekeken naar de verwerkingstijd per frame en de totale verwerkingstijd. Deze houden geen rekening met hoe lang het duurt voor de bekomen frames om weer in een videostream gestoken te worden.

Het aantal samples voor een meting is variabel, en werd gebaseerd op hoe lang het in totaal duurde voor de beeldverwerking om uit te voeren en hoe dicht dat de metingen van de CPU en GPU overeenkwamen. Metingen die sneller waren om uit te voeren of die dichter bij elkaar lagen werden vaker uitgevoerd dan trage metingen of ver uiteenliggende resultaten tussen CPU en GPU tijd.

4.1 Randdetectie (Edge Detection)



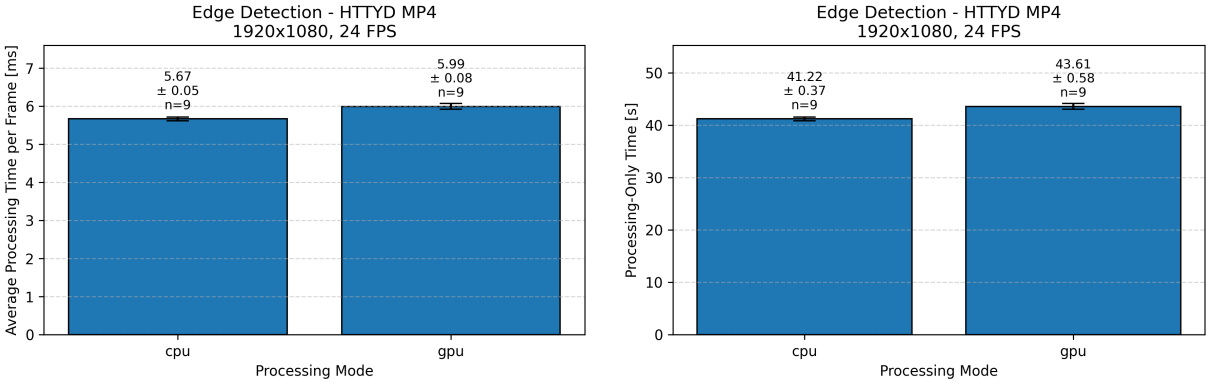
(a) Gemiddelde verwerkingstijd per frame.

(b) Totale verwerkingstijd zonder IO-overhead.

Figuur 1: Vergelijking tussen CPU- en GPU-uitvoering voor randdetectie op de Cars video.

Bij de lage workload van de Cars video (640x360) presteert de GPU duidelijk beter dan de CPU. Door de relatief beperkte resolutie blijft de hoeveelheid data die tussen CPU en GPU moet worden overgedragen klein, terwijl de parallele verwerking van de convolutiebewerkingen toch reeds voordeel oplevert.

Bij de gemiddelde workload van de HTTYD video (1920x1080) blijkt de GPU-implementatie onver-



(a) Gemiddelde verwerkingstijd per frame.

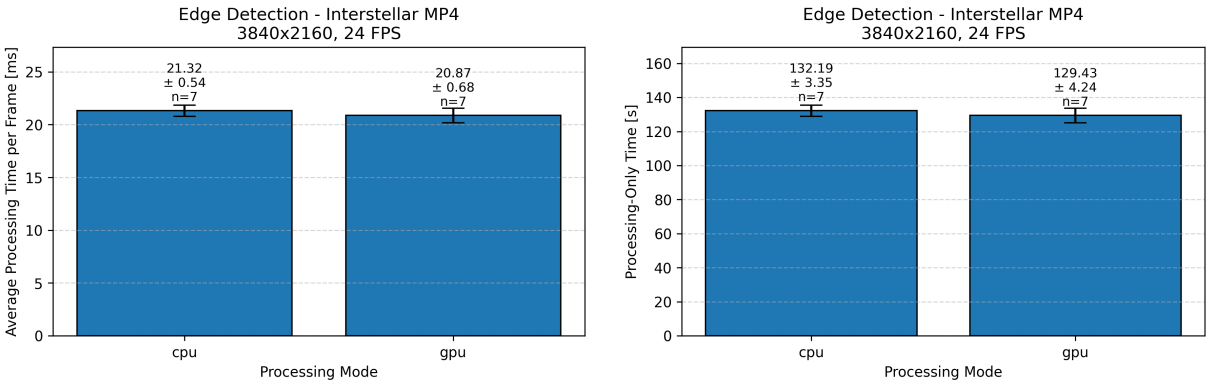
(b) Totale verwerkingstijd zonder IO-overhead.

Figuur 2: Vergelijking tussen CPU- en GPU-uitvoering voor randdetectie op de HTTYD video.

wacht licht trager dan de CPU-implementatie. De gemiddelde verwerkingstijd bedraagt ongeveer 5.67 ms per frame voor de CPU tegenover 5.99 ms voor de GPU. Over de volledige video resulteert dit in een totale verwerkingstijd van respectievelijk 41.22 s en 43.61 s.

Dit verschil kan verklaard worden door verschillende factoren. De gebruikte OpenCV CPU-implementatie is sterk geoptimaliseerd, terwijl de GPU-implementatie afzonderlijke CUDA-kernels met meerdere geheugenoverdrachten tussen host en device gebruiken. Bij een middelgrote workload kan deze overhead vergelijkbaar worden met de effectieve rekentijd van de kernels zelf, waardoor het voordeel van paralleliseren verdwijnt.

Daarnaast gebruikt de GPU-implementatie geen optimalisaties, zoals shared memory of asynchrone memory transfers. Hierdoor blijven de geheugenlatenties relatief hoog.



(a) Gemiddelde verwerkingstijd per frame.

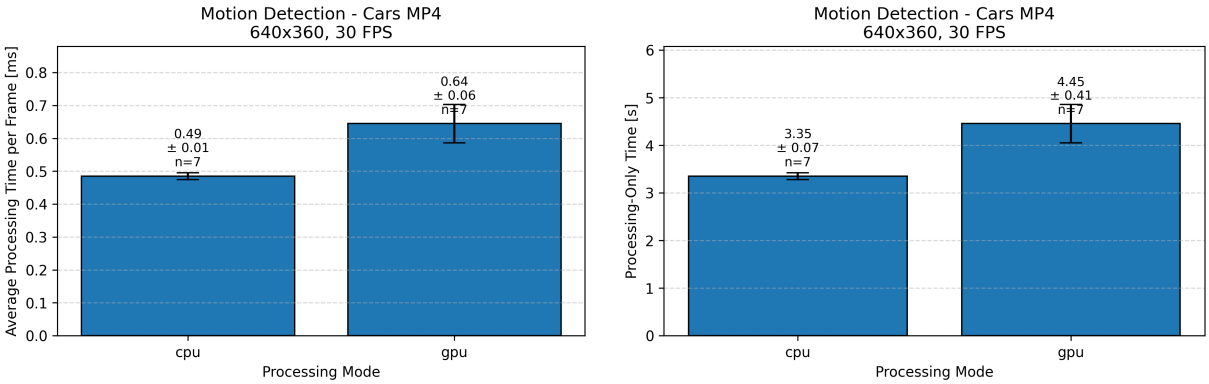
(b) Totale verwerkingstijd zonder IO-overhead.

Figuur 3: Vergelijking tussen CPU- en GPU-uitvoering voor randdetectie op de Interstellar video.

Bij de hoge workload van de Interstellar video (3840x2160) wordt opnieuw een licht voordeel voor de GPU waargenomen. De gemiddelde verwerkingstijd per frame ligt ongeveer 0.45 ms lager dan bij de CPU-implementatie. Ook de totale verwerkingstijd ligt ongeveer 2.76 s lager. Dit toont aan

dat de GPU vooral voordeel begint te bieden wanneer de hoeveelheid parallel verwerkbare data voldoende groot wordt. Naarmate de resolutie stijgt, neemt de verhouding tussen rekentijd en transfer-overhead toe, waardoor het parallelisme van de GPU beter benut wordt.

4.2 Bewegingsdetectie (Motion Detection)

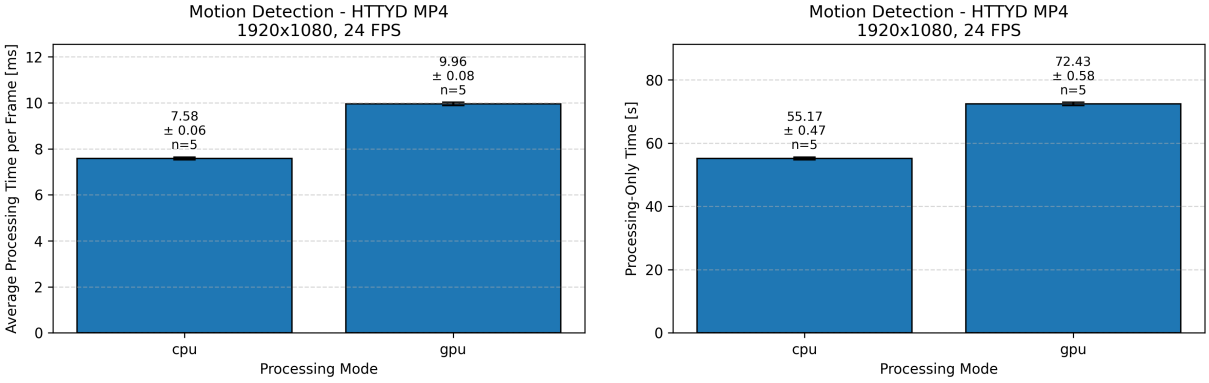


(a) Gemiddelde verwerkingstijd per frame.

(b) Totale verwerkingstijd zonder IO-overhead.

Figuur 4: Vergelijking tussen CPU- en GPU-uitvoering voor bewegingsdetectie op de Cars video.

Bij de lage workload van de Cars video (640x360) presteert de CPU beter dan de GPU voor bewegingsdetectie (motion detection). De gemiddelde verwerkingstijd bedraagt ongeveer 0.49 ms per frame voor de CPU tegenover 0.64 ms voor de GPU. Over de volledige video resulteert dit in een verwerkingstijd van respectievelijk 3.35 s en 4.45 s.

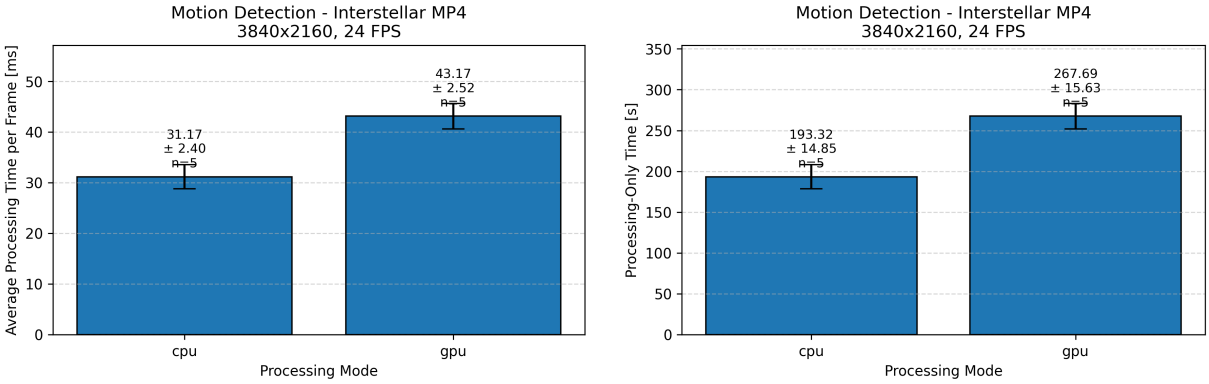


(a) Gemiddelde verwerkingstijd per frame.

(b) Totale verwerkingstijd zonder IO-overhead.

Figuur 5: Vergelijking tussen CPU- en GPU-uitvoering voor bewegingsdetectie op de HTTYD video.

Ook bij de middelgrote workload van de HTTYD video (1920x1080) blijft de CPU sneller dan de GPU. De gemiddelde verwerkingstijd bedraagt ongeveer 7.58 ms per frame voor de CPU tegenover 9.96 ms voor de GPU. Dit resulteert in een totale verwerkingstijd van respectievelijk 55.17 s en 72.43 s.



(a) Gemiddelde verwerkingstijd per frame.

(b) Totale verwerkingstijd zonder IO-overhead.

Figuur 6: Vergelijking tussen CPU- en GPU-uitvoering voor bewegingsdetectie op de Interstellar video.

Ook bij de hoge workload blijft de CPU sneller dan de GPU. De CPU-uitvoering duurt per frame 31.17 ms en de GPU-uitvoering 43.17 ms. In totaal wordt dit dan 193.32 s voor de CPU en 267.69 s voor de GPU.

Hoewel de pixelgewijze frame differencing sterk paralleliseerbaar is, bestaat de volledige bewegingsdetectiepipeline niet uitsluitend uit GPU-bewerkingen. In de implementatie wordt enkel het berekenen van het verschil tussen opeenvolgende frames uitgevoerd op de GPU. De daaropvolgende stappen, zoals morphological filtering, contourdetectie en het berekenen van de bounding boxes, gebeuren nog op de CPU met OpenCV. Hierdoor moeten de tussenresultaten telkens terug van GPU geheugen naar CPU-geheugen worden gekopieerd.

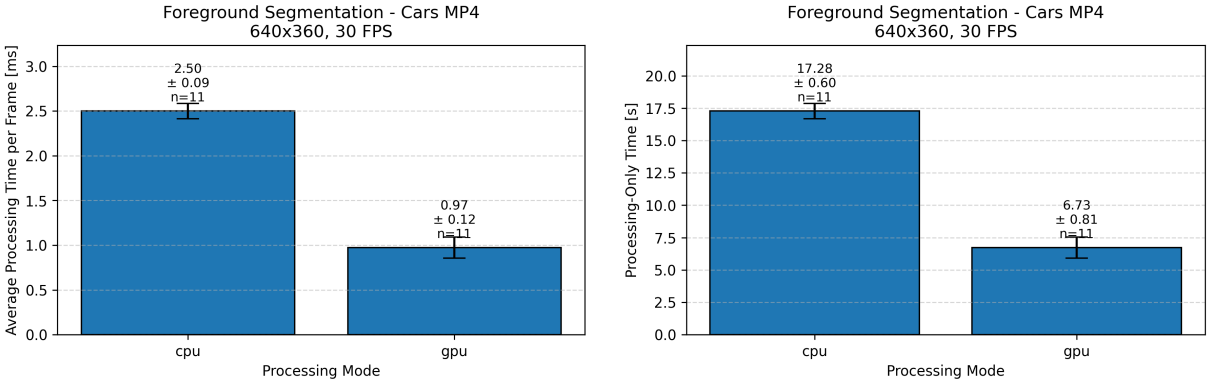
De overhead van deze geheugenoverdrachten blijkt groter dan de winst die wordt verkregen door de parallelle uitvoering van de frame differencing-kernel. Daarnaast zijn de gebruikte OpenCV-functies sterk geoptimaliseerd voor CPU-uitvoering.

De resultaten tonen aan dat GPU-versnelling vooral voordeel biedt wanneer een groter deel van de volledige verwerkingspipeline op de GPU kan blijven, zodat geheugenoverdrachten tussen CPU en GPU beperkt worden.

4.3 Voorgrondextractie (Foreground Extraction)

Bij de lage workload presteert de GPU aanzienlijk beter dan de CPU voor voorgrondextractie (foreground extraction). De gemiddelde verwerkingstijd bedraagt ongeveer 2.50 ms per frame voor de CPU tegenover 0.97 ms voor de GPU. Over de volledige video resulteert dit in een totale verwerkingstijd van respectievelijk 17.28 s en 6.73 s.

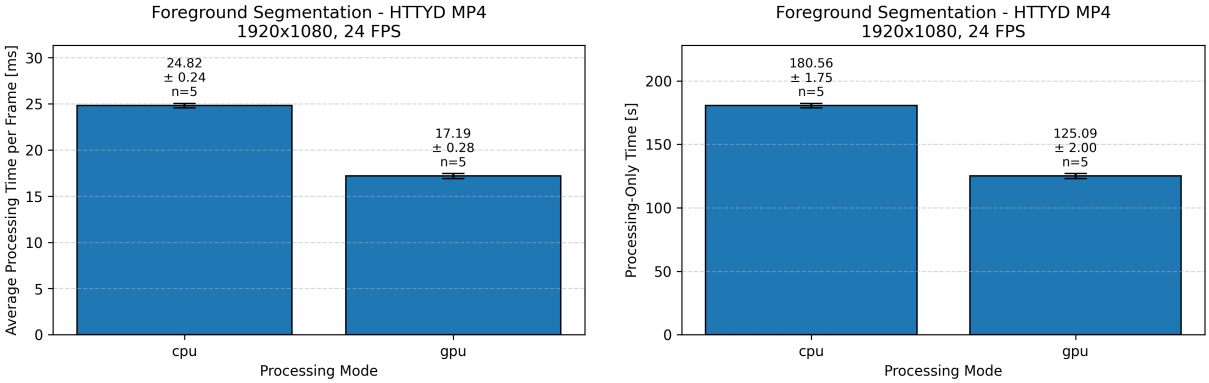
Ook bij de middelgrote workload blijft de GPU sneller dan de CPU. De gemiddelde verwerkings-



(a) Gemiddelde verwerkingstijd per frame.

(b) Totale verwerkingstijd zonder IO-overhead.

Figuur 7: Vergelijking tussen CPU- en GPU-uitvoering voor voorgrondextractie op de Cars-video.



(a) Gemiddelde verwerkingstijd per frame.

(b) Totale verwerkingstijd zonder IO-overhead.

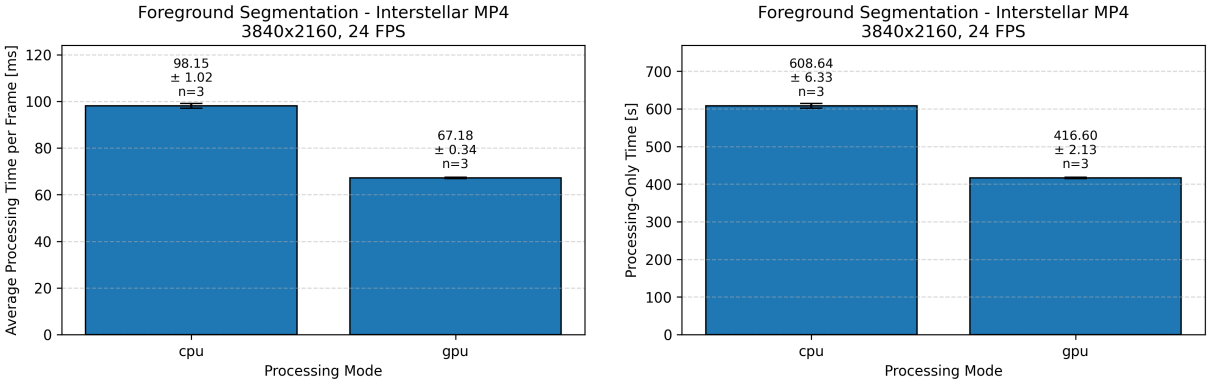
Figuur 8: Vergelijking tussen CPU- en GPU-uitvoering voor voorgrondextractie op de HTTYD-video.

tijd bedraagt ongeveer 24.82 ms per frame voor de CPU tegenover 17.19 ms voor de GPU. Dit resulteert in een totale verwerkingstijd van respectievelijk 180.56 s voor de CPU en 125.09 s.

Bij de hoge workload blijft de GPU duidelijk sneller dan de CPU. De gemiddelde verwerkingstijd bedraagt ongeveer 98.15 ms per frame voor de CPU tegenover 67.18 ms voor de GPU. Dit resulteert in een totale verwerkingstijd van respectievelijk 608.64 s en 416.60 s.

In tegenstelling tot de bewegingsdetectie wordt bij voorgrondextractie een aanzienlijk groter deel van de verwerkingspipeline op de GPU uitgevoerd. Naast het opstellen van het voorgrondmasker worden ook de Gaussian blur van de achtergrond en uiteindelijke samenvoeging van de beelden volledig op de GPU uitgevoerd.

De toegepaste Gaussian blur gebruikt bovendien een grotere kernel/matrix dan bij randdetectie, waardoor de hoeveelheid paralleliseerbare berekeningen toeneemt. Deze convolutiebewerkingen presteren goed op de GPU, aangezien elke pixel onafhankelijk verwerkt kan worden.



(a) Gemiddelde verwerkingstijd per frame.

(b) Totale verwerkingstijd zonder IO-overhead.

Figuur 9: Vergelijking tussen CPU- en GPU-uitvoering voor voorgrondextractie op de Interstellar-video.

Hierdoor wordt de overhead van geheugenoverdrachten relatief klein ten opzichte van de effectieve rekentijd van de kernels.

5 Conclusie

In dit project werden drie technieken binnen de digitale beeldverwerking onderzocht: randdetectie (edge detection), bewegingsdetectie (motion detection) en voorgrondextractie (foreground extraction). Voor elke toepassing werd een CPU-implementatie met OpenCV vergeleken met een GPU-implementatie op basis van PyCUDA.

Uit de resultaten blijkt dat het gebruik van een GPU niet direct leidt tot betere prestaties. Het behaalde voordeel hangt sterk af van het type verwerking, hoeveel parallelisme aanwezig is in de berekening, de verhouding tussen rekentijd en geheugenoverhead en of de GPU-implementatie voldoende geoptimaliseerd is en efficiënt de data kan verwerken.

Bij randdetectie bleek bij lage resoluties de overhead beperkt. Bij hoge resoluties werd de GPU efficiënt benut doordat de hoeveelheid parallel verwerkbare data sterk toenam. Voor middelgrote workloads bleef de CPU echter competitief door de sterk geoptimaliseerde OpenCV library.

Voor bewegingsdetectie presteerde de CPU in alle gevallen beter dan de GPU. Hoewel frame differencing sterk paralleliseerbaar is, werd slechts een klein deel van de volledige pipeline op de GPU uitgevoerd. De geheugenoverdrachten tussen CPU en GPU, gecombineerd met de CPU-gebaseerde morphological filtering en contourdetectie, zorgden ervoor dat de overhead groter werd dan de winst van parallelle uitvoering. Een GPU-implementatie van deze bijkomende operaties zou

een sterke verbetering zijn.

Bij voorgrondextractie werd wel een duidelijke versnelling op de GPU waargenomen. In deze toepassing werden meerdere rekenintensieve stappen volledig op de GPU uitgevoerd, waaronder de Gaussian blur en samenstelling van de afbeeldingen. Vooral bij hogere resoluties bleek de GPU hierdoor aanzienlijk efficiënter dan de CPU.

Een GPU-implementatie is dus sneller wanneer een groot deel van de volledige verwerkingspipeline op de GPU kan blijven, voldoende parallelliseerbare berekeningen aanwezig zijn en geheugenoverdrachten tussen CPU en GPU beperkt worden.

Verder werd ook duidelijk dat OpenCV sterk geoptimaliseerd is. Dit is de reden dat simpele GPU-implementaties niet noodzakelijk sneller zijn dan een CPU-implementatie.

Mogelijke uitbreidingen die de GPU-implementatie sneller zouden maken zijn het verder optimaliseren van de CUDA-kernels met technieken zoals shared memory, CUDA streams en asynchrone werking.

Dit project toont aan dat GPU-versnelling vooral interessant wordt voor computationeel intensieve beeldverwerkingstoepassingen waarbij een groot deel van de verwerkingspipeline op de GPU kan blijven.

6 Bibliografie

[1] W. contributors. Foreground detection, mar 2026.

[2] GeeksforGeeks. Webcam motion detector in python, may 2024.

[3] GeeksforGeeks. What is edge detection in image processing?, jan 2026.

FACULTEIT INDUSTRIELE INGENIEURSWETENSCHAPPEN
TECHNOLOGIECAMPUS GENT
Gebroeders De Smetstraat 1
9000 GENT, België
tel. + 32 92 65 86 10
fax + 32 92 25 62 69
iiw.gent@kuleuven.be
www.iiw.kuleuven.be

LID VAN ASSOCIATIE KU LEUVEN